DRL (WS 2025-26)

Arpitha Thimmegowda

February 8, 2026

Introduction

This project evaluates agent performance in the **Parcour** gridworld through two lenses: *computational efficiency* and *path optimization*. We measure the minimum calls to `_calc_next_state` required to first solve the task, alongside the fewest time steps needed to reach the goal across ten randomized environments. The environment was implemented following the Gymnasium API[1] and trained using Stable Baselines3[4].

1 Solving the "Parcour" Environment

1.1 Algorithm Selection: Soft Actor-Critic (SAC) [2]

For this environment, **Soft Actor-Critic (SAC)** was selected as the core algorithm. SAC is an off-policy, model-free reinforcement learning method specifically designed for continuous action spaces, making it well-suited for the Parcour environment implemented in `race_acc.py`. Its off-policy nature enables efficient reuse of past experience, which directly supports the objective of minimizing the number of `_calc_next_state` calls.

To address the exploration–exploitation trade-off, SAC employs an **entropy-regularized objective** instead of heuristic exploration strategies. This formulation improves performance by:

- **Enhanced Exploration:** Maximizing policy entropy encourages diverse action selection, helping the agent discover effective trajectories in sparse-reward settings.
- **Stable Exploitation:** Gradually reducing entropy allows the policy to converge toward high-value actions once sufficient experience has been collected.

This mechanism enables consistent learning progress while avoiding premature convergence to suboptimal policies.

Why this works:

- **Sample Efficiency:** Experience replay and off-policy updates allow the agent to learn effectively with fewer environment interactions, directly reducing the number of state transition computations.
- **Stable Convergence:** The use of twin Q-networks and a soft value update mitigates overestimation bias and improves training stability in continuous control tasks.

1.2 Hyperparameter Configuration

Through systematic hyperparameter tuning, the following values were identified as providing the most stable and efficient convergence:

Hyperparameter	**Description**	**Optimal Value**
Batch size	Mini-batch size for updates	64
Replay buffer size	Maximum replay buffer capacity	1,000,000
Learning rate	Optimizer learning rate	2.77×10^{-4}
Learning starts	Steps before training begins	4822
Gradient steps	Updates per environment step	4
Training frequency	Training interval (steps)	1
γ (Gamma)	Discount factor	0.93168
τ (Tau)	Target network update coefficient	0.02839
Entropy coefficient	Target entropy (auto)	auto_0.1
Network architecture	Hidden layer sizes	[256, 256]
Max episode steps	Episode length limit	300
Step penalty	Per-step time penalty	0.02238
Boundary penalty	Penalty for leaving valid region	0.03503
Intermediate gate bonus	Reward for passing gates	39.03
Final gate bonus	Terminal completion reward	191.87
Lookahead gates	Number of future gates considered	4

Table 1: Hyperparameter Configuration for SAC

1.3 Reward Shaping Strategy

To accelerate learning and reduce the number of environment state transitions, a structured reward shaping strategy was designed. The reward function provides both sparse task-completion signals and dense guidance to encourage efficient and stable navigation through the Parcour environment.

1. **Gate Passage Reward:[3]** A positive reward is granted whenever the agent successfully passes a gate. This guides the agent through the intended trajectory and ensures progress toward task completion.

2. **Final Gate Completion Reward:** Passing the final gate yields a significantly higher terminal reward, strongly incentivizing full episode completion rather than partial progress.

3. **Step Penalty:** A negative reward is applied at every time step to discourage unnecessary movement and promote shorter, more efficient trajectories.

4. **Boundary Penalty:** To enforce spatial constraints, a penalty is applied when the agent approaches the environment boundaries at $x = 45$ and $y = 45$. The magnitude of this penalty increases as the agent gets closer to the boundary, encouraging it to remain within the valid operational region.

5. **Distance-to-Gate Reward:[5]** A dense shaping reward is provided based on the reduction in distance to the next gate. As the agent moves closer to the upcoming gate, it receives a positive reward proportional to the decrease in distance, promoting goal-directed motion.

This combination of sparse and dense reward components ensures stable learning, faster convergence, and efficient exploration while maintaining strong guidance toward the task objective.

1.4 Results and Performance Evaluation

The agent's performance was evaluated by averaging results over **10 independent training runs**, with each model trained for a total of **150,000 environment timesteps**.

The *calls until first solve* denote the number of `_calc_next_state` calls required to solve the environment for the first time, reflecting learning speed. The *calls after first solve* represent the additional interactions performed after the initial solution, indicating training stability. The *minimum episode steps* correspond to the shortest successful episode, capturing policy efficiency.

Metric (Averaged over 10 instances)	**Value**
Total training timesteps	150,000
Mean calls until first solve	31,095.8
Mean calls after first solve	118,843.1
Mean minimum episode steps	113.7

Table 2: Average performance metrics across 10 independently trained models

1.5 Failed Approaches

Several preliminary approaches were tested before finalizing the reward strategy:

- **Boundary Penalty Only:** Training with just boundary penalties failed, as the agent received no positive feedback for progress and never passed any gates.
- **Reduced Acceleration:** Encouraging smoother motion alone did not guide the agent toward goals, resulting in random exploration.
- **Sparse Final Reward Only:** Rewarding only the final gate was too sparse, making successful episodes extremely rare.

These experiments highlighted the necessity of **reward shaping**, combining sparse completion rewards with dense guidance to accelerate learning.

2 Custom Continuous Control Environment

2.1 Environment Overview

A custom continuous control environment was designed to satisfy the requirement of creating an original Gymnasium-compatible environment with continuous states and actions. The environment models a 2D navigation task where an agent must reach a goal efficiently under non-linear dynamics. The environment includes realistic physics effects and provides visualization, plotting, and manual control capabilities.

2.2 State and Action Spaces

The observation space is a 4-dimensional continuous vector:

$$[x, y, v_x, v_y]$$

representing the agent's position and velocity.

The action space is continuous and 2-dimensional:

$$[a_x, a_y]$$

corresponding to thrust forces applied in the horizontal and vertical directions.

2.3 Environment Dynamics

The environment evolves with a fixed time step and incorporates:

- Gravity effects derived from terrain gradients
- Velocity-dependent friction and drag
- External disturbances (e.g., wind)
- Direct force-based control through continuous actions

State updates follow standard physics-based integration of forces into velocity and position.

2.4 Reward Design

The environment initially suffered from sparse rewards, making learning ineffective. To address this, reward shaping was introduced:

- Positive reward for reducing distance to the goal
- Penalty for excessive control effort
- Large terminal reward upon reaching the goal

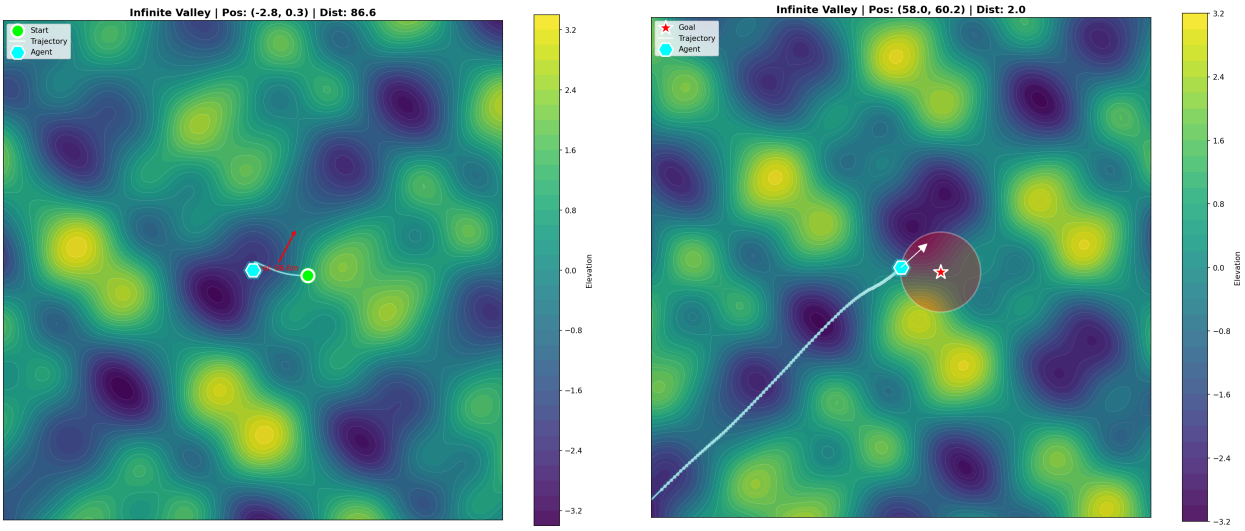
This shaping was necessary to enable stable and sample-efficient learning.

2.5 Results

The environment was trained using Soft Actor-Critic (SAC) for a total of **300,000 environment steps**. The agent first solved the task at global step **12,248**, and the best-performing policy reached the goal in a minimum of **223 steps**.

- The agent reliably reaches the goal from varied initial conditions
- The learned policy adapts to terrain variations and external disturbances
- Training remains stable due to dense reward shaping

These results demonstrate that the custom environment is well-posed, solvable, and suitable for continuous-control reinforcement learning.



(a) Environment

(b) Solved environment

Figure 1: Solved environment with Soft Actor Critic

References

[1] Farama Foundation. Gymnasium: A standard api for reinforcement learning environments. `https://github.com/Farama-Foundation/Gymnasium`, 2023.

[2] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[3] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, 1999.

[4] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. `https://github.com/DLR-RM/stable-baselines3`, 2021.

[5] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.